

Hamna - Bilingual Security Video Guide

Sinhala action guide + English speech + code segments + test evidence

Member	R.H.F. Hamna (Hamna)
Original website	http://localhost:5174/login
Secure website	http://localhost:5173/login
Assigned scope	V-07, V-08, V-09, V-12, V-15 and V-18 - configuration, transport, dependencies and CI

Common Sinhala guide

මුලින් මේ පොදු setup එක බලලා browser එක සූදානම් කරන්න

Demo setup සහ record කරන පොදු ක්‍රියා

- Original website: http://localhost:5174/login.
- Secure website: http://localhost:5173/login.
- Username: cashier.colombo. Local demo password එක login field එකට type කරන්න; password එක video එකේ නොපෙන්වන්න.
- Browser tabs දකෙක් නියාගෙන Ctrl + Tab භාවිතා කර Original සහ Secure tabs අතර මාරු වන්න.
- හැම action එකකටම cursor එක field/button/result එක උඩට ගෙන තත්පර දකෙක් නවත්වලා screenshot ගන්න.
- මුලින් Original result එක පෙන්වන්න; පස්සේ ඒ bug එක කියන්න; ඊළඟට Secure result, code segment සහ test result පෙන්වන්න.
- Live database එකට අනවශ්‍ය destructive action කරන්න එපා. Test database, code සහ existing evidence භාවිතා කරන්න.

Video introduction එකේ කියන්න

මමෙ video එකේදී මම Pubudu Tire Management System එකේ මට assign කරලා තිබෙන security workstream එක පෙන්වනවා. හැම vulnerability එකකටම මුලින් Original version එකේ අවදානම පෙන්වනවා. ඊට පස්සේ Secure version එකේ කළ fix එක, අදාළ code segment එක සහ automated test evidence එක පෙන්වනවා. Password, token, secret හෝ personal account information screen එකේ පෙන්වන්නේ නැහැ.

English introduction - say this

"Hello. In this section I will demonstrate my assigned security workstream in the Pubudu Tire Management System. For each vulnerability, I will first show the Original behavior, explain the security impact, and then show the Secure behavior, the relevant code change, and the automated test evidence. Passwords, tokens, secrets and personal account information are kept hidden."

V-07 - Sinhala guide first, English speech underneath

Sinhala guide - මමේ කතෘටස කරන විදිහ

Video steps

Original invalid request එකක raw error/stack detail පෙන්වන්න. Secure exception handler, safe JSON response, production debug setting සහ configuration test පෙන්වන්න. Screenshot names: V07-01-original-error-leak.png, V07-02-secure-safe-error.png, V07-03-error-handler-test.png.

V-07 - errors, logs සහ source disclosure

Original API එකේ error responses raw SQL/framework/stack details හමු කළ හැකි වුණා. වැරදි input එකකට 500 දමමින් internal model/class/path names පිටට යාම attackerට system එක තේරුම් ගැනීමට උදව්. Safe response එකේ userට අවශ්‍ය message/status පමණයි; developerට අවශ්‍ය detail server log එකේ පමණයි. app/Exceptions/Handler.php හා controllers හි error handling වෙනස් කර safe JSON contract එකක් කළා. .env.example production-safe defaults හා backend .htaccess hardening ද ඇත.

ඉතිරි සීමාව: Production web server document-root project directory එකට වැටී තිබේ නම් public/ වන නිවැරදිව මාරු කිරීම hosting panel access අවශ්‍යයි. ඒ නිසා "source disclosure 100% infrastructure level එකෙන් විසඳා අවසන්" කියන්න එපා; NF-5 බලන්න.

Flow එක

```
Bad request / missing record / internal exception
→ Laravel exception handler
→ safe HTTP status + generic JSON to browser
→ technical details server log ■■■
```

tests/Feature/Security/ConfigurationTest.php හි unhandled error හා missing record මගින් internals නතරපනෙන බව test කරයි.

English speech - read this in the video

V-07 - Error, log and source disclosure

[Show an Original error response with the internal details visible.]

"The Original API could return raw SQL, framework and stack information when an error occurred. These details help an attacker learn table names, class names and application paths. The browser should receive a safe status and generic JSON message, while technical details belong in server-side logs."

[Show the Secure exception handler, safe defaults and configuration test.]

"The Secure handler uses one safe JSON error contract and keeps technical details on the server. Production debug defaults are disabled, and source and secret files are protected by deployment hardening. The tests verify that unhandled errors and missing records do not disclose internal implementation details."

Key code segments and file locations

Safe exception response

File: demo/secure/pubudu-pos-api-secure/app/Exceptions/Handler.php Lines: 85-165

```
return response()->json([
    'success' => false,
    'message' => 'An unexpected error occurred. Please try again later.',
    'reference' => $reference,
], 500);
// Technical details stay in server logs.
```

Production-safe debug default

File: demo/secure/pubudu-pos-api-secure/.env.example Lines: 11-23

```
APP_ENV=production
APP_DEBUG=false
```

V-08 - Sinhala guide first, English speech underneath

Sinhala guide - මම කතාට ස කරන විදිහ

Video steps

Original CORS wildcard config සහ unknown-origin response එක පෙන්වන්න. Secure allow-list config එක සහ unknown/configured origin tests පෙන්වන්න. Screenshot names: V08-01-original-cors-wildcard.png, V08-02-secure-cors-allowlist.png, V08-03-cors-test.png.

V-08 - CORS wildcard

CORS යනු browser එකක script එක වනෙන් origin එකක API response කියවීමට අවසර දෙන policy එක. Original config/cors.php හි allowed_origins => ['*'] තිබුණා. ඔබගේ website origin එකකට API cross-origin response කියවීමට browser අවසර දිය හැකි වුණා. CORS එක login/authorization වනුවට යොදාගත නොහැක; curl/server-to-server requests CORS මගින් නවත්වන්නේ නැහැ, එහෙත් trusted frontend origins පමණක් allow කිරීම defence-in-depth.

```
'allowed_origins' => array_values(array_filter(
    array_map('trim', explode(',', (string) env('FRONTEND_URLS', '')))
)),
'supports_credentials' => false,
```

FRONTEND_URLS හි comma-separated approved origins. එය හිස් නම් wildcard එකකට fallback නැහැ - fail closed. Methods/headers ද API භාවිතා කරන දේට සීමා කළා. ConfigurationTest.php: unknown origin Access-Control-Allow-Origin නොලැබේ; configured frontend origin allow; cross-origin credentials false.

English speech - read this in the video

V-08 - Wildcard CORS policy

[Show the Original CORS configuration and an unknown-origin request.]

"The Original API allowed every origin with a wildcard CORS policy. CORS controls whether a browser script may read a cross-origin response. It is not authentication, but a wildcard policy increases the number of websites that can interact

with the API from a browser."

[Show config/cors.php and the CORS tests.]

"The Secure configuration reads an explicit comma-separated allow-list from the environment and fails closed when it is empty. Unknown origins do not receive the allow-origin header, while the configured frontend origin is accepted. Credentials are disabled for this bearer-token design."

Key code segments and file locations

Allow-list CORS origins and fail closed

File: demo/secure/pubudu-pos-api-secure/config/cors.php Lines: 37-69

```
'allowed_origins' => array_values(array_filter(
    array_map('trim', explode(',', (string) env('FRONTEND_URLS', '')))
)),
'supports_credentials' => false,
```

V-09 - Sinhala guide first, English speech underneath

Sinhala guide - මම කතාකරන ක්‍රියා විදිහ

Video steps

Original response headers සහ Secure response headers compare කරන්න. HSTS plain HTTP response එකම නොයන බව cursor එකෙන් පෙන්වන්න. Screenshot names: V09-01-original-headers.png, V09-02-secure-headers.png, V09-03-header-test.png.

V-09 - backend response headers

app/Http/Middleware/SecurityHeaders.php global middleware එකක් ලෙස app/Http/Kernel.php හි තිබේ. Global කියන්නේ normal route response පමණක් නොව 404/429/error response ද cover කිරීමයි. යවන headers:

| Header | අදහස | |---| | X-Content-Type-Options: nosniff | JSON එක HTML ලෙස browser අනුමාන නොකරයි | | X-Frame-Options: DENY | API/error page frame නුළු නොදමයි | | Content-Security-Policy | API context එකට default-src 'none' ආකාරයේ දැඩි policy | | Referrer-Policy: no-referrer | API URL/record IDs වන site එකට නොයවයි | | Cache-Control: no-store, private | Sensitive response shared cache වලට නොයවයි | | Strict-Transport-Security | HTTPS request වල පමණක් යවයි |

Theory: HSTS local plain HTTP response එකට යවන්නේ නැහැ; TLS නැති development server එකම එය අර්ථ වර්තනයයි. Header එකක් යැවීම නැතිව vulnerability fix එකක් නොවේ; V-13 XSS සඳහා output escaping, V-01 auth සඳහා route middleware තව අවශ්‍යයි.

```
if ($request->secure()) {
    $response->headers->set(
        'Strict-Transport-Security',
        'max-age=31536000; includeSubDomains; preload'
    );
}
```

මෙය SecurityHeaders.php හි condition එක. secure() true නම් පමණක් HSTS යන්නේ.

ConfigurationTest.php හි HTTP response එකට HSTS නොයන බවත් normal/error responses වල ඉතිරි headers යන බවත් බලයි.

V-09 - Missing security response headers

[Show the Original response headers.]

"The Original API responses did not consistently include browser security headers. Without these headers, browsers have fewer instructions to prevent framing, content-type confusion, referrer leakage and caching of sensitive responses."

[Show SecurityHeaders.php and the configuration tests.]

"The Secure application applies headers globally, including error responses. It sends nosniff, frame protection, a strict API content-security policy, a no-referrer policy and no-store caching. HSTS is added only when the request is already HTTPS because HSTS must not be sent over plain HTTP."

Key code segments and file locations

Conditional HSTS and security headers

File: demo/secure/pubudu-pos-api-secure/app/Http/Middleware/SecurityHeaders.php Lines: 30-88

```
$response->headers->set('X-Content-Type-Options', 'nosniff');
$response->headers->set('X-Frame-Options', 'DENY');
$response->headers->set('Cache-Control', 'no-store, private');
if ($request->secure()) {
    $response->headers->set('Strict-Transport-Security', 'max-age=31536000');
}
```

V-12 - Sinhala guide first, English speech underneath

Sinhala guide - මමේ කතාට ස කරන විදිහ

Video steps

Before composer/npm audit files, updated audit files සහ CI workflow එක පෙන්වන්න. Package install commands live දිගට type නොකර prepared output භාවිතා කරන්න. Screenshot names: V12-01-before-audit.png, V12-02-after-audit-zero.png, V12-03-dependency-ci.png.

V-12 - outdated packages සහ advisories

Original backend Laravel 10 + dependencies වල Composer advisories 41ක්; frontend npm advisories 23ක්. Package advisory කියන්නේ ඒ version එකට අදාළ known security issue එකක්. composer.json/composer.lock upgrade කර Laravel 12 line එකට ගෙන ගියා; frontend package.json/package-lock.json හි affected packages update කළා. Spreadsheet parser හා PDF library ද සැලකිලිමත් වුණා. Dependency update එකෙන් security benefit තිබුණත් breaking changes නිව්ය හැකි නිසා tests/build එක අවශ්යයි.

Evidence: evidence/before/composer-audit.txt, evidence/after/composer-audit.txt, evidence/before/npm-audit.txt, evidence/after/npm-audit.txt. දැනට saved output අනුව 41 0 සහ 23 0. මගේ "අනාගතයේ කිසි vulnerability එකක් නෑ" කියන පොරොන්දුවක් නොවේ; package advisories කාලය සමඟ වෙනස් වෙයි.

V-12 - Outdated dependencies and known advisories

[Show the before Composer and npm audit evidence.]

"The Original dependency audit reported 41 Composer advisories and 23 frontend npm advisories. These are known issues in package versions, so the application inherits their risk even when our own source code is unchanged."

[Show the updated lock files, after audit output and CI workflow.]

"The Secure project upgrades the supported framework and affected packages, then verifies the result with Composer audit, npm audit, tests and a production build. The recorded evidence changes from 41 to zero and from 23 to zero. This means no known advisory was reported by those scans at that time; it is not a promise that future advisories cannot appear."

Key code segments and file locations

Composer audit is a backend CI gate

File: demo/secure/pubudu-pos-api-secure/.github/workflows/security.yml Lines: 106-112

```
- name: composer audit
  run: composer audit --no-interaction
```

npm audit is a frontend CI gate

File: demo/secure/pubudu-pos-front-end-secure/.github/workflows/security.yml Lines: 70-75

```
- name: npm audit
  run: npm audit --audit-level=high
```

Before and after evidence

File: evidence/before/composer-audit.txt and evidence/after/composer-audit.txt Lines: lines 1-end in each evidence file

```
Before: 41 Composer advisories
After: 0 Composer advisories
```

```
Before: 23 npm advisories
After: 0 npm advisories
```

V-15 - Sinhala guide first, English speech underneath

Sinhala guide - මමේ කතාට පසු කරන විදිහ

Video steps

Original vercel.json API proxy target එකමේ http:// පෙන්වන්න. Secure file එකමේ https:// target එක පෙන්වා transport risk explain කරන්න. Screenshot names: V15-01-original-http-proxy.png, V15-02-secure-https-proxy.png, V15-03-transport-code.png.

V-15 - frontend proxy එකෙන් plain HTTP

web/vercel.json නුල /api/:path* rewrite target එක original ලෙස http://... නිවුණා. Frontend එක browserට HTTPS නිවුණත් proxy සිට APIට යන ඊළඟ hop plain HTTP නම් bearer token හා customer/sales data ඒ hop එකමේ encrypt නොවමේ. Fix එක API target https://... කිරීම. මෙය transport security. HTTPS certificate/hosting setup deployment එකමේ නව verify කළ යුතුයි.

V-15 - Plain HTTP API proxy hop

[Show the Original frontend proxy target.]

"The Original frontend was served over HTTPS, but its API proxy forwarded requests to an HTTP target. The browser-to-frontend connection could be encrypted while the next hop carried bearer tokens and business data in plain text."

[Show the Secure `vercel.json` rewrite target.]

"The Secure proxy uses an HTTPS API target. This protects the transport between the deployed frontend proxy and the API, subject to the deployment certificate and hosting configuration being valid."

Key code segments and file locations

Encrypted API proxy hop

File: `demo/secure/pubudu-pos-front-end-secure/vercel.json` Lines: 18-22

```
{
  "source": "/api/:path*",
  "destination": "https://pubudutyres.codexpress.codes/api/:path*"
}
```

V-18 - Sinhala guide first, English speech underneath

Sinhala guide - මමේ කොටස කරන විදිහ

Video steps

Original frontend response headers සහ Secure frontend CSP/security headers browser Network panel එකතේ පෙන්වන්න. Screenshot names: `V18-01-original-frontend-headers.png`, `V18-02-secure-csp-headers.png`, `V18-03-frontend-header-config.png`.

V-18 - frontend headers සහ CSP

`web/vercel.json` හි frontend pages සඳහා Content-Security-Policy, HSTS, nosniff, frame/referrer/permissions policies එක් කළා. Backend JSON API policy එක සහ frontend SPA policy එක වෙනස්; frontendට තම scripts, fonts, API connection අවශ්‍යයි. CSP එක V-13 XSSට අමතර ආරක්ෂාවක්; unsafe HTML interpolation නවම escape කළ යුතුයි. Browser DevTools Network response headers තුළ මෙය පෙන්විය හැක.

V-18 - Frontend security headers and CSP

[Show the Original frontend response headers.]

"The Original frontend did not provide a complete browser security-header policy. A frontend SPA needs a policy that permits its own scripts, fonts and API connection while blocking unsafe framing and unwanted capabilities."

[Show the Secure frontend headers in `vercel.json` and the browser Network panel.]

"The Secure deployment adds Content Security Policy, HSTS, no-sniff, frame, referrer and permissions policies. This is defence in depth. CSP does not replace output escaping for XSS, and backend authorization is still required for

protected data."

Key code segments and file locations

Frontend response security headers

File: demo/secure/pubudu-pos-front-end-secure/vercel.json Lines: 28-45

```
{
  "key": "Content-Security-Policy",
  "value": "default-src 'self'; ...; frame-ancestors 'none'"
},
{
  "key": "X-Content-Type-Options",
  "value": "nosniff"
}
```

TESTING - Sinhala guide first, English speech underneath

Sinhala guide - මේ කොටස කරන විදිහ

Video steps

Backend security suite, frontend tests, security lint, build, audit සහ GitHub Actions workflow පෙන්වන්න. 0/14 කියන්නේ attacks 14කින් exploit successful 0 කියන එක බව පැහැදිලි කරන්න. Screenshot names: TEST-01-backend-suite.png, TEST-02-frontend-suite.png, TEST-03-runtime-zero.png, TEST-04-ci-workflow.png.

Tests කියවන්නේ කොහොමද?

Before evidence: attack ■■ original code ■■■ ■■■■■■■■.
After runtime suite: attacks 14■■ ■■■■■■ ■■ ■■■ 0.
PHPUnit: positive/negative security assertions 90 tests pass.
Vitest: frontend unit tests 19 pass.

"0/14" කියන්නේ tests 14 fail කියන එක නොවෙයි. එය 14 attack attempts අතරින් exploit හැකි ගණන 0 කියන එක. PHPUnit/Vitest pass counts වනෙම. මයෙ viva එකේ පැහැදිලිව කියන්න.

Local tutorial (test database පමණක්)

Backend D:\ass_sd\work\pubudu-pos-api-secure හි .env.testing/phpunit.xml dedicated database එකට point වනෙ බව තහවුරු කර:

```
Set-Location 'D:\ass_sd\work\pubudu-pos-api-secure'
php artisan test --testsuite=Security
composer audit
```

Frontend D:\ass_sd\work\pubudu-pos-front-end-secure:

```
Set-Location 'D:\ass_sd\work\pubudu-pos-front-end-secure'
npm run lint:security
npm test
npm run build
npm audit
```

CI screen එකේ workflows greenද කියා GitHub Actions හි බලන්න. Local pass වුණා කියා CI pass වී ඇති බව නොකියන්න; repository Actions page එකේ සැලැ status බලන්න. migrate: fresh වැනි destructive command production database එකකට run කරන්න එපා.

Video එකේ පතේවන්න

1. config/cors.php wildcard ඉවත් වූ තැන සහ unknown-origin test.
2. SecurityHeaders.php global middleware; HTTPS condition සහ response headers.
3. vercel.json API target https:// සහ frontend CSP.
4. Backend 90/230, frontend 19 pass සහ after runtime Still exploitable: 0 summary.
5. GitHub Actions/security workflow files. Actions run status සාර්ථක නම් පමණක් green run එක පතේවන්න.

English speech - read this in the video

Testing, CI and the meaning of 0/14

[Show the security test suite and the CI workflows.]

"The security suite turns the findings into repeatable gates. The backend security suite records 90 passing tests and 230 assertions after the team additions, while the frontend test suite records 19 passing tests. The CI workflows run PHPUnit, security lint, frontend tests, a production build, dependency audits and secret checks on pushes and pull requests."

"The runtime summary showing Still exploitable: 0 does not mean that fourteen tests failed. It means that zero of the fourteen attack attempts remained successful after the fixes. The unit and integration tests are reported separately, and a failing CI job blocks the merge."

[Show the five deliberately deferred items only if required by the assignment report.]

"The report also clearly marks the items we did not fix in this iteration, including encrypted searchable customer data, a future httpOnly-cookie architecture, admin MFA, removal of the second UI framework and historical hostname removal. These are documented limitations, not hidden defects."

Key code segments and file locations

Security-specific CI steps

File: demo/secure/pubudu-pos-front-end-secure/.github/workflows/security.yml Lines: 43-54

- name: Security lint
run: npm run lint:security
- name: Unit tests
run: npm test
- name: Production build
run: npm run build

Backend security suite

File: demo/secure/pubudu-pos-api-secure/.github/workflows/security.yml Lines: 85-92

- name: Security regression suite
run: php artisan test --testsuite=Security
- name: Full test suite
run: php artisan test

Conclusion

Sinhala conclusion

මම memberගේ assigned work එකේදී Original bug එක, Secure fix එක, code evidence එක සහ test result එක පැහැදිලිව පෙන්වන්න. නමුත් ඇත්තටම කළ contribution එකට පමණක් claim කරන්න. Team-level test counts නමුත් individual count ලෙස කියන්න එපා.

English conclusion - say this

"To conclude, I compared the Original behavior with the Secure behavior, explained the impact of each finding, and connected each fix to source code and automated test evidence. The controls are enforced at the correct boundary, and the remaining limitations are documented rather than hidden. Thank you."

Recording safety

- Passwords, bearer tokens, client secrets and personal Google accounts must stay hidden.
- Use code and test evidence for destructive or unsafe attack steps.
- Keep the cursor on the exact result or code line being discussed.